

IG.com Homepage Migration - Developer Guide

Date: September 15, 2026

Adobe

Table of Contents

IG.com Homepage Migration - Developer Guide	3
Status: LIVE	3
Quick Reference	3
Architecture Overview	4
Local Development Setup	6
Design System	7
Blocks Reference	8
Universal Editor: Model / build:json Workflow	10
Import Pipeline & JCR Generation	10
Common Development Tasks	11
Environments	11
Git Workflow	11
Troubleshooting	12
Resources	12

IG.com Homepage Migration - Developer Guide

This guide is the technical handover for the IG.com homepage migration to AEM Edge Delivery Services. It documents the live site, architecture, design system, custom blocks, the Universal Editor model workflow, and the import/JCR pipeline. It reflects the **current** state of the codebase, including the post-go-live changes (site registration, the columns signature-detection approach, and external image-link handling).

Status: LIVE

The site is live in production:

Resource	URL
Live	https://main-iggroup-agnesl-adobe.aem.live/
Preview	https://main-iggroup-agnesl-adobe.aem.page/
Code Repository (Config Service)	agnesl-adobe/iggroup (GitHub — https://github.com/agnesl-adobe/iggroup)
AEM Author (content source)	https://author-p179457-e1900808.adobecloud.com (program p179457-e1900808)
Cloud config	/conf/iggroup
Local Dev	http://localhost:3000

The earlier **go-live blocker** is **RESOLVED**. iggroup's Edge Delivery Services site registration was completed on the author environment p179457-e1900808 (cloud config </conf/iggroup>), and both **preview** and **publish** were run. The homepage maps to the site root `/` (see `paths.json`: `/content/iggroup/language-masters/en -> /`).

Known issue (not a code defect): IG's own images served from `a.c-dn.net` and `www.ig.com/content/dam` are **hotlink-blocked (HTTP 403)** when requested from the `aem.live` origin. Those assets must be **re-hosted** (or migrated into the project DAM) before they will display on the live site. The block code handles the markup correctly; this is purely an asset-hosting restriction on IG's CDN.

Quick Reference

Resource	URL
Code Repository	agnesl-adobe/iggroup (from Config Service)
Preview	https://main-iggroup-agnesl-adobe.aem.page/
Live	https://main-iggroup-agnesl-adobe.aem.live/
Local Dev	http://localhost:3000
Config Service org	agnesl-adobe
Site name	<code>iggroup</code>

This is a **standard single-site setup** — iggroup has its own dedicated GitHub repository (confirmed via the Config Service sites/iggroup.json code object). The agnesl-adobe org hosts several other unrelated sites, but each has its own code source, so **no --pages-url flag is required** for aem up.

This is an **AEM authored (xwalk / Universal Editor) project** — content is authored in AEM (Cloud Service author p179457-e1900808) and delivered through bin/franklin.delivery, not from Document Authoring or a SharePoint/Google mount.

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+), no framework
- CSS3 with custom properties (design tokens)
- No build step for delivery — files are served directly
- Content authored in **AEM Universal Editor** (crosswalk/xwalk), delivered via franklin.delivery
- A JSON build step exists for the Universal Editor component models (see “UE Model / build:json Workflow”)

Project Structure

```

├── blocks/                # Block implementations (8 custom homepage blocks + boilerplate)
├── models/                # Universal Editor component model SOURCES (_component-*.json)
├── component-*.json       # GENERATED UE definitions/models/filters (do not hand-edit)
├── scripts/
│   ├── aem.js             # Core library (DO NOT MODIFY)
│   ├── scripts.js         # Custom page decoration, E-L-D orchestration
│   ├── delayed.js         # Adobe Target + social (tweetable) links
│   ├── utils.js           # Shared utilities (language, paths, DM, tags)
│   ├── dom-helpers.js     # Hyperscript-style element builders (div, a, img, ...)
│   ├── editor-support*.js # Universal Editor in-context editing support
│   └── at-lsig.js         # Adobe Target at.js library (loaded via delayed.js)
├── styles/
│   ├── styles.css         # Critical styles + design tokens (imports brand.css)
│   ├── brand.css          # IG.com brand tokens + Matter SQ web font
│   ├── fonts.css          # Font @font-face declarations (lazy)
│   └── lazy-styles.css    # Non-critical styles
├── tools/importer/        # Import pipeline (parsers, transformers, page-templates.json)
├── jcr/                   # Generated JCR XML + installable FileVault content packages
├── icons/                 # SVG icons
├── fonts/                 # Local font files (Adobe Clean, Roboto)
├── paths.json             # Path mappings (language-masters/en -> /)
├── fstab.yaml             # Mount point -> franklin.delivery
├── head.html              # <head> injection (aem.js, styles.css, Target/Launch, DM
└── viewers)

```

Three-Phase Loading (E-L-D)

Edge Delivery Services uses a strict Eager-Lazy-Delayed loading strategy to achieve a Lighthouse score of 100. scripts/scripts.js orchestrates all three.

Phase	Purpose	What Loads	Performance Impact
Eager (loadEager)	Render above-the-fold content as fast as possible (LCP)	styles/styles.css (+ brand.css via import),	Blocks LCP — keep minimal

Phase	Purpose	What Loads	Performance Impact
		decorateMain, first section, waitForFirstImage	
Lazy (loadLazy)	Load remaining page content after first paint	Remaining sections/blocks, header, footer, fonts.css, lazy-styles.css	Runs after Eager — safe for non-critical UI
Delayed (loadDelayed)	Load non-essential third-party scripts	scripts/delayed.js (Adobe Target via at-lsig.js, tweetable social links)	Runs ~3s after load — never blocks rendering

Rules for developers: - **Never add third-party scripts to scripts.js** — they block LCP. Use delayed.js. - **Never load fonts eagerly** — fonts.css is loaded lazily. - Blocks in the first section load eagerly; all others load lazily (automatic, based on DOM position). - Header/footer load in the lazy phase via loadHeader() / loadFooter(). - head.html currently loads Adobe Launch (assets.adobedtm.com, async), the Scene7 responsive image library (defer), and the DM Video Viewer. These are the only render-path third-party tags; keep new tags out of the eager path.

Key Files & How They Connect

File	Role	Connects To
scripts/aem.js	Core library — loadBlock, loadCSS, decorateBlock(s), decorateSections, createOptimizedPicture, loadHeader/loadFooter, sampleRUM, fetchPlaceholders, getMetadata. DO NOT MODIFY.	Imported by scripts.js and blocks
scripts/scripts.js	Entry point — E-L-D orchestration (loadEager → loadLazy → loadDelayed), buildAutoBlocks, decorateMain, moveInstrumentation (UE), isAuthorEnvironment, experimentation wiring	Imports from aem.js, utils.js, dom-helpers.js, experiment-load.js
scripts/delayed.js	Delayed phase — loads Adobe Target (loadAT via at-lsig.js) and builds <tweetable> social share links	Called by scripts.js in the delayed phase
scripts/utils.js	Shared helpers — language/path resolution, Dynamic Media URL processing, tag processing, localized-URL computation	Imported by blocks (header, hero-product, cards-markets) and scripts.js
scripts/dom-helpers.js	Hyperscript element builders (div, a, img, span, button, picture, source)	Imported by scripts.js, delayed.js, header

File	Role	Connects To
styles/styles.css	Critical CSS + design tokens; @import url('brand.css') at the top	Referenced by all blocks/pages
styles/brand.css	IG brand tokens (--brand-*) + Matter SQ @font-face	Imported by styles.css; overrides the var(--brand-*, fallback) defaults
blocks/{name}/{name}.js	Block logic — export default function decorate(block)	May import aem.js, utils.js, scripts.js (moveInstrumentation)
models/_component-*.json	UE model sources ; merged by npm run build:json into the root component-*.json	Consumed by the Universal Editor

Execution flow: page load → scripts.js → loadEager() (decorateMain + first section) → loadLazy() (remaining blocks, header, footer, fonts.css, lazy-styles.css) → loadDelayed() (loads delayed.js → Adobe Target).

Local Development Setup

Prerequisites

- Node.js (LTS; project has no pinned .nvmrc — use current LTS)
- AEM CLI: npm install -g @adobe/aem-cli

Setup Steps

```
git clone https://github.com/agnesl-adobe/iggroup
cd iggroup
npm install
aem up
```

Local Server

- URL: http://localhost:3000
- Auto-reload on file changes
- Uses preview content by default

Because this is a **standard single-site** setup with its own repo, aem up works without extra flags. (No --pages-url needed — that is only required for repoless multi-site repos.)

Linting

```
npm run lint      # runs both JS and CSS lint
npm run lint:js   # eslint . --ext .json,.js,.mjs
npm run lint:css  # stylelint blocks/**/*.css styles/*.css
```

Design System

The brand layer lives in **styles/brand.css**, imported at the top of **styles.css**. **styles.css** defines two **:root** blocks: a set of **--brand-*** fallbacks, then the semantic tokens that reference them via **var(--brand-*, fallback)**. **brand.css** overrides the **--brand-*** values with the real IG.com brand, so re-skinning is centralized in one file.

Fonts — Matter SQ

The IG brand font is **Matter SQ**, served from IG's CDN (**a.c-dn.net**) via **@font-face** in **brand.css** at weights 300/400/500/600/700, all with **font-display: swap**. **head.html** includes **<link rel="preconnect" href="https://a.c-dn.net" crossorigin>** to speed this up. All font-family tokens (**--body-font-family**, **--heading-font-family**, **--light/extra/black-font-family**) resolve to "Matter SQ", arial, sans-serif.

Local font files (fonts/: Adobe Clean variants, Roboto) remain from the boilerplate but the active brand typography is Matter SQ.

Brand Colors (**brand.css**)

Token	Value	Usage
--brand-background-color	#fff	Page background
--brand-dark-color	#000	Dark surfaces
--brand-light-color	#f5f5f5	Light surfaces
--brand-text-color	#000	Body text
--brand-light-text-color	#fff	Text on dark
--brand-link-color	#065bdb	Links (IG blue)
--brand-link-hover-color	#0446aa	Link hover
--brand-theme-color	#e61e1e	IG accent red
--brand-nav-background-color	#fff	Header background
--brand-nav-text-color	#000	Header text
--brand-footer-background-color	#000	Footer background
--brand-footer-text-color	#fff	Footer text
--brand-accent-panel-color	#c7d9fd	IG trust-stats accent panel

Semantic tokens in **styles.css** (**--background-color**, **--text-color**, **--link-color**, **--main-accent-color**, **--nav-***, **--footer-***) all resolve from these **--brand-*** values.

Typographic & Spacing Tokens (selection, from **styles.css**)

- Body sizes: **--body-font-size-m**: 22px, **-s**: 19px, **-xs**: 17px
- Heading sizes (mobile-first base): **--heading-font-size-xxl**: 32px down to **-xs**: 16px
- Spacing scale: **--spacing-tiny** (0.25rem) ... **--spacing-xhuge** (6rem)
- Radii: **--border-radius-none** ... **--border-radius-x-large** (2rem)
- Layout: **--global-section-padding**: 5rem, **--grid-margin**: 4.5rem, **--global-gutter**: 1.5rem

Breakpoints

This project uses **modern range-syntax media queries** (not the classic min-width boilerplate values). Actual breakpoints in `styles.css`:

Query	Purpose
@media (width >= 768px)	Tablet
@media (width >= 900px)	Desktop (primary nav/layout switch; <code>isDesktop</code> in <code>header.js</code>)
@media (width >= 1024px)	Large desktop
@media (width <= 768px) / (width <= 1024px)	Down-level overrides
@media print	Print styles

Section Styles

Sections support style classes and a data-attribute spacing scale: - `.section.light`, `.section.highlight` - Data-driven spacing: `.section[data-sec-spacing="section-{none|tiny|xxsmall|...|xhuge}"]` and matching `data-sec-spacing-bottom` variants. - Homepage-specific section styles used in content: **accent** (trust-stats section, uses `--brand-accent-panel-color`) and **grey** (Join IG section).

Blocks Reference

Eight custom blocks were built for the homepage (all others in `blocks/` are unmodified boilerplate and are intentionally not documented). Filtering was done by git history — only blocks with real feature commits are included.

Block	Purpose	Variants / Key Features
hero-product	Product-intro hero: centered heading/copy, CTA row, product image below	Content-classification decorate (media vs prose); external image-link handling (see below); marks 2nd+ CTA as <code>.secondary</code> ; sets first image eager/ <code>fetchpriority=high</code>
carousel-prices	Live-prices ticker — horizontally scrolling market tiles	Per row: optional flag image + name/price/change; colors gains blue, losses red by inspecting the change sign
cards-markets	Market-category grid (icon + title + description)	Responsive grid; moveInstrumentation for UE; external image-link handling on the icon cell; <code>createOptimizedPicture</code> at 400px
cards-stats	Statistics row (large figure + label)	Evenly-spaced row, wraps on small screens; moveInstrumentation for UE
columns-promo	Two “Learn more” promo panels	See “Columns Variants” — rendered via base <code>columns</code> block + signature detection
columns-feature	Benefit checklist + screenshot	See “Columns Variants”
columns-links	Audience columns of link-lists	See “Columns Variants”
columns-steps	Numbered “Join IG” steps	See “Columns Variants”

Plus a heavily-customized **header** block (11 commits): language-aware navigation, mobile/desktop menu (`isDesktop` at 900px), language switcher (`SUPPORTED_LANGUAGES`, `computeLocalizedUrl`), navigation JSON via `navigation.js`, author-environment detection.

Columns Variants — signature detection (IMPORTANT)

This supersedes any earlier note about the columns variants being “forked blocks tracked separately.” There are two locations, and only one renders on the live site:

1. **blocks/columns-promo/, columns-feature/, columns-links/, columns-steps/** — these forked-block folders still exist in the repo, but they are **NOT what renders on the live homepage**. The variants were authored as forked blocks, but md2jcr serializes them to AEM's native **columns** component, so **no variant class survives to JCR**. Do not rely on these folders for live styling.
2. **blocks/columns/columns.js + blocks/columns/columns.css** — this is the **authoritative** implementation for live render. The base columns block contains `tagIgColumnsVariant()`, which detects each variant from a **stable content signature** and adds a `columns-<variant>-ig` class:

Signature detected	Class added	Variant
Contains an <code></code>	<code>columns-steps-ig</code>	Numbered “Join IG” steps
2+ anchors matching <code>/learn more/i</code>	<code>columns-promo-ig</code>	Two promo panels
A <code></code> whose <code></code> s contain links	<code>columns-links-ig</code>	Audience link-lists
A plain-text checklist <code></code>	<code>columns-feature-ig</code>	Benefit checklist + screenshot

The detection is **additive and safe** (it no-ops on non-IG columns). The variant **styling** lives in `columns.css` scoped to `.columns.<variant>-ig`, targeting the **native columns DOM** (`.columns-row`, `.columns-img-col`) — e.g. `.columns.columns-promo-ig .columns-row > div, .columns.columns-steps-ig ... ol > li::before`.

For future devs: to change how a columns variant looks or is detected, edit `blocks/columns/columns.js` (detection) and `blocks/columns/columns.css` (styling). Do **not** edit the `blocks/columns-*/` folders expecting the live site to change — it won't.

External Image-Link Handling (IMPORTANT)

In xwalk, an image field referencing an **external DAM asset URL** is often delivered as a **bare `<a>` link to the asset URL**, not as a `<picture>`. `hero-product.js` and `cards-markets.js` detect this and render an `<img>` instead of leaving a text link:

- A shared regex identifies image URLs: `/\.(png|jpe?g|gif|webp|svg|avif)(\?|#|$)|\/is\/image\/|jcr:content\/renditions/i` (covers file extensions, Scene7 `/is/image/`, and AEM `jcr:content/renditions`).
- **hero-product.js** — classifies each cell as media (`<picture>/<img>`) or prose. If the image cell is an image-link anchor (and has no heading), it captures the `href/title` and later renders an eager `<img>` as the product image, instead of letting the URL leak into the copy.
- **cards-markets.js** — for anchors inside `.cards-markets-icon` whose `href` matches the image regex, it replaces the `<a>` with an `<img>` (using the link's title as alt, never the raw URL text), then runs `createOptimizedPicture`.

This pattern is common for external-DAM references in xwalk; expect to reuse it in any new block that takes an image field sourced from an external DAM.

Universal Editor: Model / `build:json` Workflow

Content is authored in the **Universal Editor**. The component definitions the editor uses are **generated** from sources — never hand-edit the root `component-*.json` files.

- **Sources** live in `models/`: `_component-models.json`, `_component-definition.json`, `_component-filters.json` (plus the standard `_page.json`, `_section.json`, `_button.json`, `_image.json`, `_text.json`, `_title.json`). The customized model sources include the homepage block models (card, columns, hero, etc.).
- **Build step** merges them into the repo-root files consumed by the editor:

```
npm run build:json
```

which runs (in parallel):

- `build:json:models` → `component-models.json`
- `build:json:definitions` → `component-definition.json`
- `build:json:filters` → `component-filters.json`

(Uses `merge-json-cli` + `npm-run-all`.)

Workflow: edit files under `models/` → run `npm run build:json` → commit both the sources and the regenerated root `component-*.json` → the Universal Editor picks up the new/changed models. `scripts/editor-support*.js` provides in-context editing behavior, and `moveInstrumentation` (from `scripts.js`) preserves UE instrumentation attributes when blocks rebuild the DOM (used by `cards-markets` and `cards-stats`).

Import Pipeline & JCR Generation

The homepage was migrated from `https://www.ig.com/en` using the `tools/importer/` pipeline, then converted to JCR for ingestion into AEM.

`tools/importer/`

- **page-templates.json** — the home template: the representative URL plus DOM selectors mapping source-page nodes to each of the 8 block variants.
- **parsers/** — one parser per block variant (`hero-product.js`, `carousel-prices.js`, `cards-markets.js`, `cards-stats.js`, `columns-promo.js`, `columns-feature.js`, `columns-links.js`, `columns-steps.js`) that turn source DOM into block tables.
- **transformers/** — `ig-cleanup.js` (strips source cruft) and `ig-sections.js` (establishes section boundaries / section styles).
- **import-home.js** + **import-home.bundle.js** — the runnable import script (bundled form used by the importer UI/CLI). `urls-home.txt` holds the input URL. `reports/` holds run output.

JCR generation (`jcr/`)

The import produces `content/en.plain.html`, which is converted to JCR XML with `@adobe/helix-md2jcr`, using this project's generated `component-models.json`, `component-definition.json`, and `component-filters.json`.

- `jcr/content/iggroup/language-masters/en/.content.xml` — the JCR representation of the migrated homepage: **11 sections**, all **8 block variants**, section styles (accent on trust-stats, grey on Join IG), and page metadata.
- **Installable FileVault packages** (JCR XML is not delivered from git — it must be installed into AEM author):

- `jcr/iggroup-en-homepage-1.0.0.zip` — the homepage content; filter scoped to `/content/iggroup`, carries the full page chain (`iggroup` → `language-masters` → `en`). Install via `/crx/packmgr` (Upload → Install) or `vlt/oakpal`.
- `jcr/iggroup-site-configuration-1.0.0.zip` — site configuration package.
- Unzipped sources live under `jcr/package-build/` and `jcr/config-package/`.

Common Development Tasks

Add a New Block

1. Create `blocks/{name}/{name}.js` and `{name}.css`.
2. Implement `export default function decorate(block)`.
3. If it takes an image field that may come from external DAM, reuse the image-link detection pattern from `hero-product.js` / `cards-markets.js`.
4. If it rebuilds DOM and must stay editable in UE, call `moveInstrumentation`.
5. Add/adjust the model in `models/` and run `npm run build:json`.
6. Test at `http://localhost:3000`, then push to a feature branch.

Style a Columns Variant

1. Edit detection in `blocks/columns/columns.js` (`tagIgColumnsVariant`).
2. Edit styling in `blocks/columns/columns.css` scoped to `.columns.<variant>-ig`.
3. Do NOT edit `blocks/columns-*/` — those do not render on the live site.

Modify Global Styles / Rebrand

1. For brand color/font changes, edit `styles/brand.css` (`--brand-*` tokens).
2. For structural/token changes, edit `styles/styles.css`.
3. Watch for CLS; test across sections.

Add Analytics/Marketing Tool

1. Add to `scripts/delayed.js` (Adobe Target is already wired there).
2. Never add to `scripts.js` (blocks performance).
3. Verify delayed loading in the Network tab.

Environments

Environment	URL Pattern	Purpose
Local	<code>http://localhost:3000</code>	Development
Feature Branch	<code>https://{branch}-iggroup-agnesl-adobe.aem.page</code>	PR testing
Preview	<code>https://main-iggroup-agnesl-adobe.aem.page</code>	Staging
Live	<code>https://main-iggroup-agnesl-adobe.aem.live</code>	Production

Git Workflow

- Branches: `feature/description`, `fix/description` (keep short — URL limits).

- PRs: include the preview URL, pass `npm run lint`, no console errors.

Troubleshooting

Homepage images show broken / 403

IG's CDN (`a.c-dn.net`, `www.ig.com/content/dam`) hotlink-blocks the `aem.live` origin. Re-host the assets or migrate them into the project DAM. Not a code bug.

A columns variant looks unstyled

Confirm `tagIgColumnsVariant()` added the `columns-<variant>-ig` class (inspect the DOM). If the content signature changed, update the detection in `blocks/columns/columns.js`. Remember the `blocks/columns-*/` folders are inert.

An image field renders as a text link

The asset was delivered as an external `<a>` link. Apply the image-link detection pattern (regex + replace with `<img>`) as in `hero-product.js` / `cards-markets.js`.

Block not loading

Block folder name must match the block class in the DOM; JS must export `default function decorate`; check the Network tab for 404s.

UE models not updating

Re-run `npm run build:json` and commit the regenerated root `component-*.json`.

Resources

Resource	URL
Edge Delivery Services Documentation	https://www.aem.live/docs/
Developer Tutorial	https://www.aem.live/developer/tutorial
Block Collection	https://www.aem.live/developer/block-collection
E-L-D Loading	https://www.aem.live/developer/keeping-it-100
Universal Editor	https://www.aem.live/developer/universal-editor-blocks
Config Service (site config API)	https://www.aem.live/docs/admin.html#tag/siteConfig